ARCHITECTURE RESEARCH REPORT

Novel Architectures in KERNOS

A code-verified survey of the mechanisms a personal agentic operating system contributes beyond the mid-2026 landscape of agentic harnesses

Version 1.1 — revised 22 July 2026 (v1.0 published 21 July 2026)

Subject codebase: KERNOS (main branch; 7,073 test functions across 391 test files, counted 22 July 2026; 76 kernel tools)

Method: three-track verification — documented claims, direct code inspection, adversarial landscape survey

Abstract

KERNOS is a single-author personal agentic operating system: a kernel that owns persistence, context assembly, capability routing, safety enforcement, and identity, wrapped around a principal reasoning agent whose "only job is to think." This report asks one question of it: *which of its architectural elements are genuinely novel in the current landscape of agentic harnesses, and which are commonplace?* Commonplace machinery — MCP tool access, platform adapters, SQLite persistence, provider fallback chains, schedulers, sandboxed code execution, event logs — is explicitly excluded. Every claim about KERNOS below was verified against its source code by independent inspection agents; every novelty judgment was tested against an adversarially-conducted survey of the mid-2026 landscape (Letta/MemGPT, Mem0, Zep/Graphiti, LangGraph/LangMem, CrewAI, AutoGen, Anthropic/OpenAI/Google native memory, SICA, Darwin-Gödel Machine, MOSS, AlphaEvolve, NeMo Guardrails, LangGraph interrupts, Claude Code permissions, Operator, Devin, Character.AI, Replika, Nomi, Alexa/Google household assistants, and the relevant 2025–2026 research literature).

The survey finds **ten element clusters that clear the novelty bar**, three of them with no located precedent of any kind: (1) fully automatic hierarchical context spaces with scoped inheritance and cross-context signaling; (2) a governed recursive self-improvement stack whose specific safeguards — boot-guard auto-rollback, database-enforced recursion bounds, constitutional-file boundaries, diff-hash-bound approval receipts — are individually absent from every surveyed system; and (3) a friction loop that combines automatic failure-pattern detection with upfront-approval rule proposals, a combination the landscape splits apart everywhere else. The remaining clusters are novel as formalizations or integrations of ideas that exist only as fragments, research prototypes, or single niche implementations elsewhere. Discrepancies between KERNOS's documentation and its code, and the limits of the survey, are reported in §6.

Contents

1 Introduction and Scope

The agentic-harness field of mid-2026 has converged on a recognizable commodity stack: an LLM loop dispatching tools over MCP, a vector or graph memory store with LLM-adjudicated fact extraction, conversation summarization at a token threshold, a permission prompt somewhere between the model and the world, and — increasingly — background memory consolidation. Frameworks differ in ergonomics far more than in architecture. Against that baseline, KERNOS is interesting not because it does these things, but because of a set of mechanisms that, on inspection, have no counterpart in the surveyed landscape.

KERNOS describes itself as "a personal intelligence kernel that serves the full breadth of one person's life." Its architectural stance is stated in its own design documents: *the kernel owns all infrastructure; the agent's only job is to think.* Behavioral rules are enforced in the dispatch path rather than requested in the prompt; memory compresses without losing truth; context is organized into auto-created domains; the system participates in its own maintenance. These are claims. This report's job was to (a) verify them against the code, (b) hold them against what the rest of the field actually ships, and (c) keep only what survives both tests.

The bar for "novel" used throughout: a mechanism qualifies if a good-faith, adversarial search of products, frameworks, and research literature found *no implementation* of it (Novel — no precedent found), found only research prototypes or a single niche implementation (Rare precedent), or found the components separately but never the integration that does the actual work (Novel as integration). Mechanisms that any mature framework ships — however well KERNOS executes them — are excluded, per the commissioning brief.

2 Method

Three independent verification tracks were run and then reconciled:

- **Claims extraction.** KERNOS's own documentation — `DECISIONS.md` (38 permanent architectural decisions), `docs/TECHNICAL-ARCHITECTURE.md`, `docs/DESIGN-PRINCIPLES.md` (17 named patterns), and the architecture corpus under `docs/architecture/` — was read in full to enumerate every claimed mechanism.
- **Code verification.** Three inspection agents independently traced each claimed mechanism to source, reporting file-and-line evidence, actual constants and formulas, and — critically — discrepancies where the code does not match the documentation. Four material discrepancies were found and are reported in §6 rather than papered over.
- **Adversarial landscape survey.** Two research agents surveyed the mid-2026 landscape with instructions to *refute* novelty wherever possible, including a dedicated verification pass hunting for counter-examples. Two refutations succeeded (a niche dual-strength memory implementation; a niche single-pass compress-and-extract library) and are incorporated below. One fabricated claim by a search-engine summarizer was caught by direct fetch of the primary source and discarded. One fetched web source contained instruction-shaped injected text; it was treated as data and not acted upon.

All KERNOS mechanisms described in §4 are code-verified unless explicitly marked otherwise. Landscape claims carry their sources in the References section; negative claims ("no system found that…") are claims about a bounded search, not proofs of absence, and §6 states the search's known blind spots.

3 What Is Deliberately Excluded as Commonplace

The following KERNOS subsystems are competently built but architecturally unremarkable in 2026, and appear nowhere in §4: MCP capability integration and the capability registry; platform adapters (Discord, Telegram, SMS) as dumb pipes; SQLite/WAL persistence with a JSON fallback; data-driven LLM provider fallback chains; cron-style and event-based scheduling; sandboxed subprocess code execution; append-only event streams for audit (standard event-sourcing); the workflow engine's descriptor/trigger/action shape (commodity workflow-automation architecture, deterministic predicates included); vector retrieval with lexical fallback; per-instance data isolation; webhook receivers; and the use of the Agent Client Protocol to talk to external CLI agents (adopting an emerging standard is good judgment, not novelty — though the surrounding consultation-audit log and reentrancy guard are unusually careful). Bi-temporal invalidate-don't-delete fact handling — KERNOS's "shadow archive" — is likewise excluded as a headline item: Zep/Graphiti is an established reference implementation of the same instinct, although KERNOS applies it more pervasively (files, covenants, members, knowledge) than any surveyed memory product.

4 The Novel Elements

4.1 Automatic Hierarchical Context Spaces

NOVEL — NO PRECEDENT FOUND FOR THE COMBINATION

HOW KERNOS HANDLES IT

Every conversation lives inside a tree of *context spaces*: General (root, depth 0) → Domain (depth 1) → Subdomain (depth 2), plus a separate System plane (`kernos/kernel/spaces.py`). Each space owns its own conversation log, its own compaction state (Ledger + Living State, §4.2), its own promoted tool set, its own posture (a working-style note injected into the prompt), and its own knowledge scope. Three properties make the design singular:

- **Spaces are created by the system, not the user.** The sole creation path is compaction-driven: after a space compacts, a cheap LLM assesses whether the compacted content constitutes a coherent domain, checks existing spaces for duplicates and drift, and creates a new space only at HIGH confidence — medium-confidence verdicts are logged as near-misses, not acted on (`handler.py:4382–4459`). The user never files anything; domains condense out of lived conversation.
- **A scope chain provides inheritance.** Memory queries, file reads, and archive searches walk *up* the parent chain — facts and files in a parent are visible from its children, and local files shadow same-named parent files. Writes default local, with ancestor-only exceptions. After a parent compacts, per-child *briefings* (3–8 durable bullet points) are generated and injected into each child's memory block.
- **Spaces talk across the tree, sparingly.** A post-turn check deposits one-time *cross-domain signals* when entities mentioned in the current space have knowledge in an unrelated domain and the turn carried a meaningful update; a *downward search* answers quick questions about another domain without switching into it (the router's `query_mode`), injecting the answer into the current space's results. Routing itself is a per-message LLM call with an explicit cost-asymmetry instruction: staying wrong is cheap, switching wrong is expensive.

THE PREVAILING SHAPES

Every shipped equivalent is manual. ChatGPT Projects, Claude Projects, Perplexity Spaces, Gemini Gems, and Copilot Notebooks are user-created containers with static instructions; OpenAI's own community forums carry open feature requests for automatic organization. Research systems have fragments: MemoChat (arXiv 2308.08239) auto-segments dialogue into topics with per-topic summaries but has no inheritance or cross-topic signaling; HiMem (arXiv 2601.06377) builds a vertical episode→knowledge hierarchy, not inheriting topic spaces; A-MEM (NeurIPS 2025) gets emergent cross-topic linkage from a flat Zettelkasten graph, not scoped containers. The landscape survey's summary judgment: automatic creation + scoped inheritance + cross-context signaling + independent per-space rolling summaries, combined, was *"the single clearest gap identified across the whole report."*

WHAT THE KERNOS SHAPE BUYS

The manual-container model externalizes the filing problem onto the user, which is precisely the labor a personal agent exists to absorb; in practice users under-create containers and everything lands in one context. Flat-memory systems (one store, retrieval does everything) avoid the filing problem but lose domain isolation: tone, tools, procedures, and working state for a D&D campaign and a medical situation genuinely should not share a context.

KERNOS's shape gets isolation without filing labor, inheritance without duplication (identity facts live once, at the root), and controlled leakage (signals are deposited, one-time, on evidence of relevance) instead of either hermetic silos or ambient bleed. The HIGH-confidence-only gate plus alias/rename tracking addresses the failure mode that would otherwise kill auto-creation — proliferation of near-duplicate spaces.

4.2 Compaction as a Multi-Harvest Metabolic Boundary

NOVEL AS INTEGRATION — COMPONENTS HAVE RARE RESEARCH PRECEDENTS

HOW KERNOS HANDLES IT

When a space's conversation log exceeds a token threshold, one compaction pass (`kernos/kernel/compaction.py`) produces, in a single LLM call:

- **A Ledger entry** — an append-only, immutable topic index line (`## Compaction #N (source: log_NNN)`) pointing back to the archived raw log. The Ledger is never edited; `remember_details(log_NNN)` retrieves the exact original text. Summaries are a lens over the archive, never a replacement for it.
- **The Living State** — a full rewrite of "what is true right now": active scene state, pending decisions, in-progress work. Explicitly not a topic summary; rewritten every cycle.
- **A fact harvest** — structured `add / update <id> / reinforce <id>` operations against the knowledge store, where `reinforce` feeds the dual-strength memory model (§4.3).
- **Follow-up extraction** — implicit commitments and deadlines (`USER_COMMITMENT`, `AGENT_COMMITMENT`, `EXTERNAL_DEADLINE`, `FOLLOW_UP`) become provisional scheduler triggers, deduplicated against existing ones and capped at a 90-day horizon.
- **Recurring-workflow capture** — a multi-step workflow the user has repeated 3+ times is surfaced for formalization.
- **Stewardship signals** — value extraction, tension detection, and sensitivity classification (open/contextual/personal) ride the same pass, by design at zero additional LLM cost.
- **An adaptive seed depth** — the compaction LLM itself decides how many trailing messages to carry into the fresh log (`SEED_DEPTH: N`, clamped 3–25, default 10): a creative scene keeps 15–20 messages of momentum; a factual Q&A keeps 3–5.

Compaction also drives space creation (§4.1) and parent→child briefings. It is not a summarizer; it is the system's metabolic cycle — the single boundary where conversation is digested into every durable substrate the kernel keeps.

THE PREVAILING SHAPES

The dominant industry pattern is two or more separate pipelines: summarize on a threshold (deterministic, fixed template or fixed carry-forward), and extract facts in a distinct pass (Mem0's extract-then-update, LangMem's separate `SummarizationNode` and memory manager, Cloudflare's compaction-then-extraction, CrewAI's per-fact calls). Anthropic's server-side compaction and Claude Code's `/compact` use threshold triggers with fixed preservation categories; context editing is deterministic FIFO clearing. On the individual components: append-only index + lossless dereference is a recognized, growing research pattern (LCM's immutable store with summary-node views, arXiv 2605.04050; Memex(RL), arXiv 2603.04257; Letta's recall memory as the 2023 precedent) but rare in shipped products; LLM-chosen adaptive carry-forward depth is a named research category (Focus Agent, AdmTree) with no flagship product implementation found; single-pass piggybacked compression+extraction was located in

exactly one niche open-source library (agentmemory), and one production memory vendor (ProMem) explicitly argues *against* it. No surveyed system combines more than one of these; none attaches follow-up scheduling, workflow capture, or sensitivity classification to the same boundary.

WHAT THE KERNOS SHAPE BUYS

Three things. *Losslessness with a working set:* because the Ledger indexes rather than replaces, compaction can be aggressive without destroying evidence — the standard summarize-and-discard shape forces a permanent trade between context size and recoverable truth. *Coherence:* one model, reading one conversation once, decides what is a fact, what is a commitment, what is sensitive, and what still matters — the multi-pipeline shape re-reads the same text with different prompts and produces artifacts that can disagree with each other. *Economics:* the marginal cost of each additional harvest is prompt lines, not LLM calls, which is why KERNOS can afford stewardship analysis that other architectures would have to schedule as a separate (and therefore usually skipped) job. The adaptive seed depth, finally, fixes a real defect of fixed-window carry-forward: the correct amount of conversational momentum to preserve is content-dependent, and only the model reading the content knows it.

4.3 Dual-Strength (Bjork) Memory

RARE — ONE CONTEMPORANEOUS NICHE PRECEDENT; ABSENT FROM ALL MAJOR PLATFORMS

HOW KERNOS HANDLES IT

Every knowledge entry carries two independent variables, after Bjork & Bjork's New Theory of Disuse: `storage_strength` (how well-established a fact is — grows by 1.0 each time compaction re-confirms it, never decays) and a *retrieval strength* computed at read time, never stored (`kernos/kernel/state.py:165–199`):

```
effective_stability = base_stability × (1 + 0.1 × ln(1 + storage_strength))
retrieval_strength  = (1 + k × days_elapsed / effective_stability)^(−0.5),  k =
0.9^(−1/0.5) − 1
```

— an FSRS-6-family power-law forgetting curve whose stability is modulated by the entry's *lifecycle archetype*: identity facts decay against a 730-day base, structural 120, habitual 45, contextual 14, ephemeral 1. Retrieval ranking multiplies relevance by this strength, so a well-reinforced fact resists displacement by a recent-but-trivial one, while an ephemeral detail fades in a day regardless of how confidently it was stated.

THE PREVAILING SHAPES

Single-variable decay is the universal paradigm: the Stanford Generative Agents recency×importance×relevance score and its descendants; CrewAI's exponential half-life (`0.5^(age/30d)`); Mem0's access-based re-rank (1.5× boost to 0.3× floor); Ebbinghaus-curve variants (YourMemory, powermem, Oblivion). Mem0's own blog cites Bjork explicitly — and explicitly states the reference is "conceptual only," not implemented. The adversarial pass located exactly one genuine implementation of the two-variable model anywhere: Vestige, a small Rust MCP memory server created January 2026, with separately persisted storage/retrieval strengths and the same FSRS-family decay — a striking independent convergence, developed contemporaneously with KERNOS's BJORK phase (April 2026). No major platform — Letta, Mem0, Zep, LangMem, CrewAI, Anthropic, OpenAI, Google — implements it.

WHAT THE KERNOS SHAPE BUYS

Single-variable decay conflates two different questions: *how established is this fact?* and *how likely is it to matter right now?* Conflating them produces the two familiar failure modes of agent memory — long-known core facts ("allergic to penicillin") getting crowded out by recency, and stale ephemera surviving because they were accessed once recently. Splitting the variables lets reinforcement and decay operate independently, and the archetype-modulated stability encodes a prior the single-variable systems cannot express: the *kind* of fact determines how it should forget. Computing retrieval strength at read time (rather than batch-updating stored scores) also eliminates an entire class of background-job staleness. KERNOS additionally wires the model into its metabolism: reinforcement events come from compaction's fact harvest (§4.2), so "the user re-confirmed this" is detected as a side effect of digestion, not by a dedicated tracker.

4.4 The Cognitive UI: A Rendered, Not Accumulated, Context Window

NOVEL AS FORMALIZATION — SECTIONED PROMPTS ARE COMMON; THE RENDERING DOCTRINE IS NOT

HOW KERNOS HANDLES IT

KERNOS treats the context window as a user interface it *renders each turn from substrate state*, not a log it appends to. One assembly phase (`kernos/messages/phases/assemble.py`) composes the agent's entire view into named zones in fixed order — RULES, ACTIONS (a cacheable static prefix), then NOW, STATE, RESULTS, PROCEDURES, MEMORY (a dynamic suffix re-rendered every turn) — ending with a generated tool-signature endcap at the recency position. Three details elevate this beyond prompt hygiene:

- **A field-provenance map** (`kernos/kernel/cognitive_context/field_provenance.py`) declares, for every field in every zone, exactly which substrate source populates it — a single seam that makes "what did the model see when it decided?" a queryable fact rather than a reconstruction.
- **Provenance tags on knowledge.** Every injected fact carries its epistemic status — `[stated]`, `[observed]`, `[established]`, `[remembered]` — so the agent can weigh a user's direct statement differently from its own inference.
- **The DEPTH contract.** The prompt tells the agent, truthfully, that its context is curated and that full retrieval stands behind it: "You are not reconstructed from summaries — you are precisely briefed for this turn with full retrieval capability behind you." This works only because it is architecturally true (§4.2's lossless archive).

THE PREVAILING SHAPES

Sectioned system prompts are informal best practice — Anthropic's context-engineering guidance recommends them while explicitly declining to standardize a block grammar. Real named-zone implementations exist as private vendor conventions (Claude Code's static/dynamic zones, per third-party documentation; Hermes Agent's three cached tiers — which are deliberately rebuilt at session start rather than per turn, a documented counter-example to per-turn rendering). No cross-vendor spec exists; nothing surveyed ships a field-level provenance map, per-fragment epistemic tags, or an explicit rendered-not-accumulated doctrine. Most harnesses still accumulate: the context is a transcript plus retrieval, and "what the model saw" is answerable only by replaying the run.

WHAT THE KERNOS SHAPE BUYS

The design's own claim is the correct analysis: an agent operates the world through its context window the way a human operates software through a screen, so per-turn choice of what to surface is interface design, not prompt engineering. The practical dividends are concrete: prompt caching falls out of the static/dynamic split rather than

being retrofitted; any zone can refresh on its own schedule; audits get a ground-truth answer to what the agent knew; and the tool-signature endcap at the recency position measurably targets the tool-fumbling failure mode (§4.11). The novelty here is honest but bounded — this is the strongest available *formalization* of a direction the field is drifting toward, not an idea without relatives.

4.5 The Quiet Cohort

NOVEL AS A NAMED DISCIPLINE — DISTINCT FROM BOTH SINGLE-LOOP AND MULTI-AGENT ORTHODOXY

HOW KERNOS HANDLES IT

One principal reasoning agent is surrounded by single-purpose, cheap-model, strict-contract micro-judgments — a message analyzer (classification + knowledge selection + preference detection in one combined call), a router, a gate evaluator, a schedule extractor, a step-completion verifier, a Messenger welfare judge, a friction observer, a fact harvester. Three disciplines are load-bearing: **selective invocation** (each cohort runs only when its signal is plausible — budget-, env-, and predicate-gated — and is omitted entirely otherwise); **fail-open** (a broken cohort never blocks the turn: the verifier defaults to complete, the analyzer degrades to no-op); and **silence** (cohort output shapes the substrate or the rendered context, but no cohort is visible to the user or to the principal agent — "the agent experiences a world that is already understood; the user experiences one mind, not a committee").

THE PREVAILING SHAPES

The field offers two poles. Single-loop harnesses push every side-judgment into the principal agent's own context — classification, safety, memory selection all compete with the actual work. Multi-agent frameworks (CrewAI, AutoGen, and their descendants) model side-work as *peer agents that converse* — heavyweight, chatty, latency-additive, and visible in the interaction. Guardrails frameworks (NeMo, Guardrails AI) are structurally closest to cohorts but are validators at I/O boundaries, not judgment workers threaded through a turn pipeline, and none articulates the invoked-only-when-plausible / fail-open / silent triple as a discipline.

WHAT THE KERNOS SHAPE BUYS

Specialist quality at commodity cost, without either pole's tax. The combined-call pattern matters economically (the analyzer resolves three concerns in one cheap invocation; covenant relevance selection added *zero* calls by extending the analyzer's schema), and the silence rule matters experientially — the user of a personal agent should never watch a committee deliberate. The fail-open rule encodes an ordering of harms most frameworks never state: a missing judgment is cheaper than a blocked conversation.

4.6 Proportional Dispatch-Time Authorization and Covenants as Architecture

NOVEL — NO FORMALIZED EQUIVALENT FOUND

HOW KERNOS HANDLES IT

Two interlocking mechanisms govern every tool dispatch (`kernos/kernel/gate.py` , 6-step evaluation):

- **User intent is authorization.** Every dispatch is classified by effect — read / soft_write / hard_write — from its *actual arguments*. A reactive soft-write (the user asked for it, this turn) executes without confirmation: the request *is* the authorization. The gate spends its judgment only where it belongs — hard writes, third-party impact, proactive/background actions, and covenant conflicts — and evaluates those proportionally to ambiguity × loss-cost (execute / confirm / clarify). Internal housekeeping is exempt in the other direction:

proportionality cuts both ways. The claim of this section is the *policy*, formalized and enforced at the dispatch boundary; the supporting machinery beneath it (single-use approval tokens bound to tool name and input hash, an amortization cache, denial-loop tracking) is competent engineering, not the novelty.

- **Covenants are first-class stored objects, evaluated in the dispatch path.** User behavioral rules ("always confirm before spending money," "never reference the surprise party") are auto-captured from conversation into typed rows — SPIRIT / MUST_NOT / MUST / PREFERENCE / ESCALATION — with scopes (global, space, member), lifecycle (supersession, never deletion), and enforcement tiers. MUST_NOT rules are checked *before* the reactive bypass can fire. Injection is selective: pinned covenants always render; situational ones only when the message analyzer deems them relevant — so accumulating rules do not bloat the prompt. Covenant violations surface to the agent as *conflicts to resolve*, not silent denials.

THE PREVAILING SHAPES

Human-in-the-loop is everywhere; the *policy of when* is nowhere. LangGraph's `interrupt()` leaves placement entirely to the graph author. Claude Code's permission modes prompt by tool category — its default mode still confirms actions the user explicitly just requested, and its `auto` mode's request-alignment check is an undocumented runtime heuristic, not a stated principle. Operator and Devin hard-code categorical checkpoints (payments, PR merge). The survey directly checked Anthropic's published agent-safety framework and the "Before the Tool Call" literature: no system states "explicit in-turn user request is sufficient authorization; gate only the proactive and the high-stakes" as a formal dispatch policy. On the covenant side, guardrails frameworks (NeMo's Colang rails) are *developer*-authored configuration, not user-declared rules captured at runtime; user-facing instruction files (CLAUDE.md, custom instructions) are prompt text — which is precisely the shape KERNOS's design documents reject: "a suggestion the model may or may not honor under context pressure." Nothing surveyed stores user rules as scoped, tiered, lifecycle-managed objects evaluated in the dispatch path with selective injection.

WHAT THE KERNOS SHAPE BUYS

Confirmation fatigue is the quiet killer of agent utility: a system that re-confirms what the user just asked trains the user to click yes, which destroys the value of the confirmations that matter. Formalizing "reactive intent is authorization" spends the user's attention only on genuine risk — and the friction observer (§4.8) even watches for violations of this policy (`GATE_CONFIRM_ON_REACTIVE` is a tracked defect signal, a policy-with-teeth detail no surveyed system has). Covenants-as-architecture, meanwhile, moves the user's standing rules from the model's mood to the kernel's law: enforcement survives context pressure, session boundaries, and model swaps, and conflict-surfacing (rather than silent denial) keeps the agent able to explain itself. The combination amounts to a coherent, articulated safety philosophy — "agent thinks, kernel enforces; conservative about what the agent initiates, not what the user requests" — where the rest of the field ships mechanisms and leaves the philosophy to the integrator.

4.7 The Governed Self-Improvement Stack

NOVEL — MULTIPLE CONSTITUENT SAFEGUARDS INDIVIDUALLY ABSENT FROM EVERY SURVEYED SYSTEM

HOW KERNOS HANDLES IT

KERNOS can rewrite its own running codebase, and nearly everything interesting about it is in the governance. The `improve_kernos` loop (`kernos/kernel/improvement_loop_workflow.py`) runs: spec drafting → external-reviewer spec review (iteration-capped) → implementation by an external coding agent in a worktree → post-implementation review with GREEN/YELLOW verdicts → a hard human approval gate (`awaiting_commit_approval`; nothing lands without an explicit yes) → commit, push, redeploy via restart → **post-restart self-test** → completion wake, bounded recovery (at most two approval-gated fix-up commits), or rollback-and-abandon. The safeguards, each code-verified:

- **Boot-guard auto-rollback** (`kernos/setup/boot_guard.py` + `start.sh`): an applied update sits on probation (`.update_pending`); after N failed boots the guard `git reset --hard`s to `.last_known_good` — promoted only on a clean startup — and files a rollback notice the agent surfaces in its own voice.
- **The unprotectable bootstrap, named.** `start.sh` runs the rollback machinery, so a bad autonomous commit to it cannot be auto-recovered; it is therefore constitutionally human-only, and the loop routes around it by construction rather than pretending the constraint away.
- **The request-fidelity gate:** the operator's *original request* is injected into the final code review, so the reviewer can fail a diff for fulfilling the spec but not the ask — latitude on *how*, fidelity on *what*. (This caught a live wrong-deliverable during operation.)
- **A recursive self-heal lane with database-enforced bounds** (`recursive_self_heal.py`): when an attempt aborts on a bug in the loop's own machinery, one bounded child repair may be spawned — but only if the failure matches a signature requiring both a deterministic *positive symptom* and a *negative guard* ruling out ordinary task failure; the child is verified *hermetically* (a temp git fixture, deterministic pass/fail, no model judgment); and the runaway bound lives in the database, not the prompt — `UNIQUE(parent, relation)` (one child per parent), `UNIQUE(root, signature, fingerprint)` (never repeat a fix per root), `child_depth ≤ 1` checked against the root, with the edge reserved transactionally *before* spawn so a restart cannot launder depth.
- **Constitutional boundaries:** an explicit file list (the approval gate, dispatch, the boot guard, the heal lane itself, guardrail tests — `recursive_self_heal.py:55–84`) such that any verified self-repair touching them routes to human review *even when it passes*, with untracked files included in the diff check so nothing slips past porcelain.
- **Diff-bound approval receipts:** durable approval records with a schema-enforced state machine (pending → approved → consumed) binding the parent commit SHA and the expected diff hash — an approval is for *this change*, not for whatever the branch contains by the time it lands.
- **Two quieter lanes on shared governors:** a daily self-maintenance review — exactly 42 single-owner functional elements covering every module, each reviewed through a corrective lens (drift/decay) and a generative lens (does this still serve the whole?), with a binding discipline of ≤1 minor reversible idea per review, signal-promoted selection under a rotation-floor coverage guarantee, and a coverage-gap fingerprint

that flags new unmapped modules; and a friction-response lane (§4.8). All three share one remediation mutex; everything terminates in the same human approval gate.

THE PREVAILING SHAPES

The self-improving-agent literature is real and fast-moving — SICA self-edits inside a Docker container with no protected files and no rollback; the Darwin-Gödel Machine rewrites its own code under human supervision in sandboxes and was observed *reward-hacking* (fabricating tool outputs), caught by archive transparency rather than prevented structurally; AlphaEvolve optimizes target code, never its own harness; MOSS (May 2026) is the closest relative, with health-probe-gated rollback for self-modified deployments, gated on human consent. The survey's findings on the specific safeguards are stark: boot-guard auto-rollback to last-known-good as a *shipped* capability — not found (closest: an unmerged OpenClaw feature request); DB-enforced recursion bounds on self-repair — not found anywhere; a formal constitutional-file mechanism — not found in any flagship system (the term of art does not exist; the nearest thing is blog-level convention); approval receipts bound to a commit SHA/diff hash — not found, and notably the underlying replay-vulnerability class is *live in mainstream tooling* (GitHub "Verified" commit signatures can currently be rewritten onto a different commit). No surveyed system runs a scheduled generative self-review of its own architecture at all.

WHAT THE KERNOS SHAPE BUYS

The research systems optimize benchmark scores in sandboxes; KERNOS's loop deploys to the process that is *currently serving its user*, which is why its governance had to be invented. The design insight that generalizes furthest is that every bound on a self-modifying system must live somewhere the modification cannot reach: recursion depth in database constraints rather than prompts (a prompt-bound is a suggestion to the very model being bounded — DGM's observed reward-hacking is the empirical case for this), rollback in a bootstrap layer the loop may not touch, approvals bound to content hashes rather than intentions. The second insight is the explicit naming of the unprotectable layer: acknowledging that recovery cannot recover itself, and making that single point human-only, is more honest than the uniform-sandbox posture of the research systems — and more actionable than their uniform human-supervision posture. The review triangle (proposer, external reviewer, design authority, human only at the gate) has demonstrated the full cycle live: the system shipped an improvement to its own improvement loop, an external reviewer found a real edge-case bug in it, and a third agent fixed it — with receipts.

4.8 The Friction Loop: Automatic Detection, Upfront Approval

NOVEL — THE LANDSCAPE SPLITS THIS COMBINATION APART EVERYWHERE

HOW KERNOS HANDLES IT

A post-turn observer (`kernos/kernel/friction.py`) watches every turn for eleven concrete failure signals — empty responses, the gate confirming a reactive action, a stated preference the parser missed, a tool available but not used, repeated provider errors, plus two *opportunity*-class signals (a better method found on retry; a deferred capability) — and writes structured diagnostic reports. Above it, a pattern accumulator (`behavioral_patterns.py`) fingerprints recurring user corrections (first 80 chars, normalized) against four thresholds (format 3, workflow 3, boundary 2, preference-drift 2). At threshold, the system classifies the pattern — *behavioral* (propose a covenant), *workaround* (flag as a system malfunction rather than papering over a code bug), or *uncertain* (surface both readings) — and proposes a durable rule **to the user, for approval, before it takes effect**. A decline is remembered and not re-proposed until three further occurrences. The reactive lane (`friction_response.py`) closes the loop on operational errors with a two-key memory —

`friction_signature` (what the problem is) × `resolution_fingerprint` (what was tried) — under a binding anti-loop rule: *never repeat a failed fingerprint for the same signature*. Fixes pass through verification states (`PENDING_VERIFICATION → resolved / recurred_failed / unknown_no_observation`) with an honest epistemic distinction: quiet-while-idle is not evidence of resolution. The human surface is one plain-language question whose "yes" is validated (same user, same space, ≤4 words, no negation, exactly one pending fix, 15-minute TTL) and consumed single-use.

THE PREVAILING SHAPES

The survey's explicit absence finding: the landscape divides cleanly into *automatic-and-silent* (Claude Code's auto-memory writes notes from corrections with no upfront approval; ChatGPT's "Dreaming" rewrites memory in the background; PRELUDE/CIPHER infer preferences and update policy unasked) and *approval-gated-but-not-automatic* (Cursor's DIY self-improving-rules pattern requires manual setup; Sierra's Ghostwriter drafts approval-gated fixes from *aggregate* enterprise analytics, not one user's lived friction). Reflexion-style self-critique is episodic and within-task — nothing consolidates into durable cross-session rules. Zero systems were found that both automatically detect correction/friction patterns *and* put each proposed durable rule through upfront user approval.

WHAT THE KERNOS SHAPE BUYS

Silent memory-writing systems accumulate rules the user never sanctioned and eventually contradict them — the user discovers what the system "learned" only when it misfires. Manual systems never fire at all, because users do not file tickets against their own assistant. The KERNOS combination gets adaptation with consent: the system does the noticing, the user does the deciding, and declines are themselves remembered. Two second-order choices deserve note: the *workaround* classification refuses to convert code bugs into behavioral rules — a category error every silent learner happily commits — and the never-repeat-failed-fingerprint rule plus verification-state honesty prevent the classic autonomous-remediation pathology of retrying the same failed fix forever and declaring quiet success.

4.9 Multi-Member Emergent Identity and Relationship-Mediated Disclosure

NOVEL — NO PRODUCTION PRECEDENT FOR ANY OF THE THREE COMPONENTS

HOW KERNOS HANDLES IT

One KERNOS instance serves a household of members, and three unusual decisions govern it:

- **One hatched agent per member, not per install.** "Kernos" is the platform, not the agent. Each member's agent *hatches* through an organic early-conversation arc — engagement guidance covering entry energy, resonance testing, self-naming ("the agent names itself when the moment is right, not on turn 1"), curiosity, and honesty about uncertainty — graduating via LLM consolidation into a durable personality profile (`member_profiles`: agent_name, emoji, personality_notes, communication_style). An `inherit` mode lets a household opt into copies of the first agent instead. (Instrumentation caveats in §6.)
- **Relationships are declared, not inferred.** Pairwise, bidirectional rows with permission profiles (verified in code as a three-level matrix — no-access / by-permission / full-access — gating request-action, inform, and ask-question intents), conservative provisional defaults until both sides confirm, and topic exceptions expressed as covenants with relationship scope. All knowledge is sensitivity-classified at write time (open / contextual / personal, defaulting personal when unsure) as part of the compaction harvest.

- **A welfare judgment sits above the permission matrix.** Every permitted cross-member exchange passes through the Messenger cohort (`cohorts/messenger.py`), a cheap-model judgment on whether the exchange serves the *disclosing* member's welfare, returning pass / revise / refer. The design example is exact: your spouse can see your calendar — the relationship grants it — but the system still exercises judgment about the therapy appointment. Permission is necessary; it is not sufficient.

THE PREVAILING SHAPES

Household assistants (Alexa, Google Home) are one persona with per-user *data* access. Companion products are single-relationship by design (Replika's explicit "AI monogamy"), or fixed-persona (Character.AI characters are the same for everyone; Meta's celebrity personas are shared). Nomi.ai markets organically evolving personalities but per-companion-per-user, continuous rather than milestone-gated. Hermes Agent requires a separate instance per personality. Norton's 2026 family assistant declares family relationships but documents no disclosure mechanics. The research literature confirms the underlying problem is real and unsolved — LLM agents leak secrets through social contagion at 20–45% rates in large-scale simulation; ConfAIde shows GPT-4-class models revealing secrets in up to 93% of complex-tier cases — and no relationship-permission system was found tested against it, let alone shipped. The survey's verdict on all three components: no confirmed production precedent.

WHAT THE KERNOS SHAPE BUYS

The shared-instance/distinct-identity combination is the shape a family actually needs: shared substrate (one calendar reality, one household knowledge base, cross-member coordination) with individual relationships (a child's agent and a parent's agent should not be the same character, and neither should know everything the other knows). Declared-not-inferred relationships put the disclosure boundary under explicit human control where inference would fail silently and catastrophically. The Messenger is the most philosophically interesting piece: it encodes the observation that access-control matrices cannot express *kindness* — some disclosures are permitted, true, and still wrong — and it prices that judgment at one cheap-model call on the rare cross-member path. Sensitivity-classification-at-write-time (rather than at disclosure time) means the judgment about how private a fact is gets made when context is richest, not when the request arrives.

4.10 Token-Budgeted Tool Surfacing

NOVEL MECHANISM — THE CATEGORY IS EMERGING, THIS SHAPE IS ABSENT

HOW KERNOS HANDLES IT

Tool exposure is a budgeted economy, not a static registry. A universal catalog holds every tool (kernel, MCP, workspace-built) with a version counter. Per turn, surfacing runs in three tiers: **Tier 1**, no LLM — kernel tools + common MCP tools + the space's *local affordance set* + session-loaded tools, covering ~80% of turns; **Tier 2**, an intent-based LLM scan over the full catalog when Tier 1 is insufficient; **Tier 3**, promotion — a successfully used uncommon tool enters the space's local affordance set and is Tier-1 next turn. Promotion is space-scoped with a bloat guard (domain tools cannot promote into the root space; only tools marked universal can), and each space lazily rescans the catalog for newly registered tools relevant to its domain when the version counter moves. The full set is held under a token-budgeted, schema-weighted LRU window with pinned and active zones, so the standing tool surface competes for context like any other resource — and the per-space affordance set means a D&D space and a finance space converge on different resident toolkits over time.

THE PREVAILING SHAPES

Static full enumeration remains the MCP default — the protocol itself requires it at session init; dynamic loading is an open, unadopted spec discussion. Retrieval-based selection is emerging but effectively single-vendor: Anthropic's Tool Search Tool (beta, deferred loading, ~85% token reduction in its own eval) and Claude Code's auto-switch above a context threshold; research parallels (RAG-MCP, MCP-Zero, Toolshed) remain unshipped. The survey found *no* implementation anywhere that treats the standing tool-definition set as a token-budgeted LRU cache with usage-driven promotion — the closest adjacent work evicts tool-call *history*, not the tool set — and none with per-context resident-toolkit promotion.

WHAT THE KERNOS SHAPE BUYS

Retrieval-per-turn treats tool selection as a stateless search problem; KERNOS treats it as a *locality* problem, which it empirically is — the tools a space needed yesterday predict what it needs today. The promotion economy converts each successful Tier-2 search into future Tier-1 hits, so the system's common case gets cheaper with use (zero selection-LLM calls on ~80% of turns), while the bloat guard stops the root context from becoming the junk drawer every long-lived agent accumulates. This also composes with §4.4's deeper claim: under a rendered-context doctrine, an unsurfaced tool is a control the agent does not have this turn — so surfacing policy is not an optimization but the definition of the agent's action space, and budgeting it explicitly is the honest form of that decision.

4.11 Smaller Original Mechanisms

ORIGINAL FRAMINGS — NARROWER IN SCOPE, INCLUDED FOR COMPLETENESS

- **The 24 Escalation.** Unknown-sender abuse handling escalates each failed attempt through a geometric ladder of time units — 24 seconds → 24 minutes → 24 hours → 24 days → 24 years → 24 centuries — with acceleration for attempts made *while blocked*, entirely via pre-rendered responses at zero LLM cost (the tiers are literal constants in `instance_db.py`). Flat thresholds plus exponential backoff dominate the API layer; graduated bans exist in platform account moderation; the specific legible ladder with the acceleration sub-rule was not found as a named pattern anywhere. A live-operations lesson is encoded alongside it: internal senders bypass the ladder entirely, because the system once rate-limited its own self-directed plan.
- **Narration-audited completion.** An autonomous plan step counts as complete only when its *named actions* ran, not when the turn produced text: one cheap strict-contract call audits the step description against the agent's own report, and a named deficit re-dispatches the same step once, carrying the deficit. This targets a lie every multi-step framework tells itself ("turn ended" ≠ "work done") — observed live twice in KERNOS's own self-tests before the fix — and is cheaper than re-deriving ground truth because the receipts culture (§4.4, §4.7) makes the agent's narration trustworthy enough to audit. Reflexion-lineage self-critique is the nearest relative and is episodic, within-task, and advisory rather than completion-gating.
- **The typed failure that is its message.** `ToolFailure` subclasses `str`: a tool's returned failure *is* its error message, so every legacy string consumer keeps working byte-for-byte while dispatch boundaries detect the type and record `is_error=True` with a failure code and a pre-side-effect retry-safety flag. Zero-migration failure visibility is an elegant engineering move rather than an architecture; it earns its line here because invisible returned-failures (recorded as successes, so plans complete over them) are a live defect class across the field.
- **The plain-English self-test.** A checklist the agent executes against itself end-to-end through its own tools — no delegation, no destructive actions, honest PASS/PARTIAL reporting, receipts for every claim — testing the

integration no unit suite reaches (model × prompt × tools × substrate). KERNOS's dispatch-reliability stack was largely driven by failures this surfaced.

5 Comparative Matrix

ELEMENT	KERNOS MECHANISM	CLOSEST LANDSCAPE ANALOGUE	ASSESSMENT
Context partitioning	Auto-created hierarchical spaces; HIGH-confidence gate; scope-chain inheritance; cross-domain signals; parent briefings; per-space compaction	Manual containers everywhere (ChatGPT/Claude Projects); research fragments (MemoChat, HiMem, A-MEM) each with one property, never combined	Novel — clearest gap found
Compaction	Single-pass Ledger + Living State + fact/follow-up/workflow/sensitivity harvest + LLM-chosen seed depth; lossless index into raw archives	Two-stage summarize-then-extract everywhere; lossless-index and adaptive-depth exist separately as research (LCM, Focus Agent); one niche single-pass library	Novel as integration
Memory decay	Two-variable Bjork model; FSRS-family power law; archetype-modulated stability; read-time computation; compaction-driven reinforcement	Single-variable decay universal (Generative Agents, CrewAI, Mem0); one niche contemporaneous implementation (Vestige)	Rare — one precedent
Context assembly	Rendered-not-accumulated; named zones; field-provenance map; epistemic tags; static/dynamic cache split; DEPTH contract	Sectioned prompts as informal practice; private vendor conventions (Claude Code, Hermes); no provenance map or doctrine anywhere	Novel as formalization
Auxiliary judgment	Quiet cohort: selectively invoked, fail-open, silent micro-judgments around one principal agent	Single-loop (everything inline) or conversing peer agents (CrewAI, AutoGen); I/O validators (NeMo)	Novel as discipline
Authorization policy	"Reactive intent is authorization" formalized; effect classification on actual arguments; proportional ambiguity × loss-cost; covenants as dispatch-path objects with selective injection	HITL placement left to developers (LangGraph); categorical checkpoints (Operator, Devin); developer-authored rails (NeMo); no formalized policy found	Novel
Self-improvement governance	Boot-guard rollback; DB-enforced recursion bounds; constitutional file boundaries; SHA/diff-hash-bound approvals; fidelity gate; post-restart self-test; 42-element daily self-review	Sandboxed research systems (SICA, DGM); MOSS's consent-gated health-probe rollback; each specific safeguard individually absent	Novel — multiple absences
Behavioral learning	Automatic friction/correction detection + upfront-approval rule proposals + remembered declines + anti-loop fix memory	Automatic-and-silent (Claude auto-memory, ChatGPT Dreaming) or approval-gated-but-manual (Cursor DIY, Sierra aggregate); never both	Novel — combination absent
Multi-user identity	Per-member hatched agents in one instance; declared pairwise relationships with permission profiles; Messenger welfare judgment above permissions; write-time sensitivity classes	One-persona-many-users (Alexa, Meta); single-relationship companions (Replika, Nomi); problem confirmed unsolved in research (ConfAIde)	Novel — no precedent
Tool surfacing	Three-tier surfacing with usage-driven per-space promotion; schema-weighted token-budgeted LRU window; bloat guard	Static enumeration (MCP default); single-vendor retrieval (Anthropic Tool Search); no budgeted-LRU tool window found anywhere	Novel mechanism
Abuse escalation †	Geometric time-unit ladder with acceleration-while-blocked, zero-LLM	Flat thresholds + backoff (APIs); count-based moderation tiers (platforms)	Original framing (§4.11 — not counted)
Step completion †	Narration-audited completion gating plan progress	Reflexion-lineage self-critique (episodic, advisory)	Original framing (§4.11 — not counted)

† The final two rows are §4.11 "Smaller Original Mechanisms" — minor original framings included for completeness. They are *not* counted among the ten element clusters of the headline finding, which correspond to §4.1–§4.10.

6 Limitations, Discrepancies, and Integrity Notes

DOCUMENTATION-VS-CODE DISCREPANCIES FOUND DURING VERIFICATION

- **The "0.10 strength filter" documented for memory retrieval was not found in code.** Filtering rides embedding-similarity thresholds (0.65 semantic, 0.5 lexical); dual strength modulates *ranking*, not a hard gate. The §4.3 analysis rests only on what the code does.
- **Hatching's documented "15-turn minimum" is a `graduation_threshold = 25` in code**, and the "8 engagement points" exist as prompt-level guidance rather than instrumented, individually tracked signals. §4.9 accordingly presents hatching as a designed conversational arc with partial instrumentation, and the multi-member novelty claim rests principally on the per-member-identity *architecture*, the relationship matrix, and the Messenger — all fully verified.
- **Spirit-first rendering** is implemented as ordering within the covenant listing rather than a distinguished prompt position.
- **Covenant enforcement is deliberately decentralized** (gate + reasoning path + assembly-time injection) rather than a single enforcement module; the "kernel enforces" claim holds, but as a property of the dispatch path, not of one file.

SURVEY LIMITATIONS

- Negative findings are bounded by search effort. The hierarchical-spaces gap survived an adversarial re-check but with an explicit incompleteness caveat (some products and newer startups unchecked). "No precedent found" should be read exactly that way.
- The two refuting counter-examples (Vestige for dual-strength memory; agentmemory for single-pass compress+extract) rest on direct source inspection of small repositories by one research agent; one popularity metric attached to the latter could not be corroborated. Both were treated as genuine precedent for novelty-downgrade purposes — the conservative direction.
- Some comparative claims about closed products (ChatGPT memory internals, Claude Code compaction templates) are sourced from third-party reverse engineering and are flagged as lower-confidence in the underlying survey.
- During research, one web source returned instruction-shaped injected text and one search summarizer fabricated a taxonomy attributed to a vendor document; both were caught (the latter by direct fetch of the primary source) and excluded.

SCOPE LIMITATIONS OF THIS REPORT

- KERNOS is a single-author system. The test-suite figure on the cover (7,073 test functions across 391 files) was counted directly from source on 22 July 2026 but the suite was not independently executed for this report; the v1.0 snapshot cited ~5,181 passing tests from project records, and the suite has grown since. Mechanisms verified here were verified as *present and implemented*, not benchmarked for efficacy against alternatives.
- Several described subsystems ship default-off or behind flags (friction-response lane, recursive self-heal, the decoupled-cognition path); the report includes them as implemented architecture while noting their activation status where material.

- Novelty is assessed against the surveyed landscape as of July 2026; the field's velocity makes every such assessment perishable.

7 Conclusion

Strip away everything commonplace, and KERNOS still holds a substantial residue — which is itself the notable result, since for most agentic harnesses the same operation leaves nothing. The residue is not scattered: it traces to three convictions applied with unusual consistency. First, *structure over instruction* — wherever the field writes prompt text (behavioral rules, safety policy, recursion limits, completion criteria), KERNOS builds substrate: covenants in the dispatch path, bounds in database constraints, approvals bound to diff hashes, completion gated on audited receipts. Second, *one metabolism* — compaction as the single boundary where conversation becomes memory, schedule, structure, and self-knowledge at once, where the field runs parallel pipelines that disagree. Third, *proportionality* — safety spent where loss-cost lives rather than uniformly, attention charged for what it displaces, judgment work priced at the cheapest model that can carry it.

The strongest individual claims, in order: the governed self-improvement stack (several safeguards with no located implementation anywhere, addressing a vulnerability class demonstrably live in mainstream tooling); automatic hierarchical context spaces (the survey's single clearest gap); the friction loop's automatic-detection-plus-upfront-approval combination (a square the landscape leaves empty from both sides); and the multi-member identity/disclosure architecture (no production precedent, against a research literature that confirms the problem unsolved). The dual-strength memory and the compaction integration follow, tempered honestly by rare precedents. The Cognitive UI and Quiet Cohort matter less as inventions than as the discipline that makes the rest composable — and disciplines, unlike mechanisms, are the part of this system other harnesses could adopt tomorrow.

References

1. KERNOS project documents: `DECISIONS.md`; `docs/TECHNICAL-ARCHITECTURE.md`; `docs/DESIGN-PRINCIPLES.md`; `docs/kernos-introduction.md`; source under `kernos/` (all mechanisms cited by file above).
2. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560; Letta memory-blocks and sleep-time-compute documentation (docs.letta.com; arXiv:2504.13171).
3. Mem0: arXiv:2504.19413; "Introducing Memory Decay in Mem0" (mem0.ai, May 2026). Zep/Graphiti: arXiv:2501.13956.
4. Anthropic: memory tool, context editing, and compaction documentation (platform.claude.com); "Effective Context Engineering for AI Agents" (Sep 2025); Advanced Tool Use / Tool Search Tool (anthropic.com/engineering); Claude Code memory and permissions documentation (code.claude.com).
5. OpenAI: ChatGPT memory documentation and "Dreaming" announcement (openai.com); Operator announcement. Google: Gemini Personal Intelligence (blog.google, Jan 2026).
6. LangGraph persistence and interrupts; LangMem SDK (docs.langchain.com; langchain-ai.github.io/langmem). CrewAI memory documentation (docs.crewai.com). AutoGen memory protocol and discussion #5006 (microsoft.github.io/autogen).
7. Park et al., "Generative Agents," arXiv:2304.03442. MemoChat arXiv:2308.08239; HiMem arXiv:2601.06377; A-MEM arXiv:2502.12110; LCM arXiv:2605.04050; Memex(RL) arXiv:2603.04257; Focus Agent arXiv:2601.07190; AdmTree arXiv:2512.04550; ProMem arXiv:2601.04463.
8. Self-improving systems: SICA arXiv:2504.15228; Gödel Agent arXiv:2410.04444; ADAS arXiv:2408.08435; Voyager arXiv:2305.16291; Darwin-Gödel Machine arXiv:2505.22954 (sakana.ai/dgm); Huxley-Gödel Machine arXiv:2510.21614; AlphaEvolve arXiv:2506.13131; MOSS arXiv:2605.22794; Ratchet arXiv:2605.22148; Reflexion arXiv:2303.11366; PRELUDE/CIPHER arXiv:2404.15269.

9. Safety: NVIDIA NeMo Guardrails, Guardrails AI, Meta LlamaFirewall (arXiv:2505.03574); Anthropic agent-safety framework (anthropic.com/news); Devin checkpoints (docs.devin.ai); "Before the Tool Call," arXiv:2603.20953; GitHub Verified-commit signature rewriting (thehackernews.com, July 2026).
10. Multi-user: Replika (single-relationship design); Character.AI persona documentation; Nomi.ai; Norton Family Assistant (June 2026); "Got a Secret? LLM Agents Can't Keep It," arXiv:2605.27766; ConfAIde, arXiv:2310.17884.
11. Tool surfacing: RAG-MCP arXiv:2505.03275; MCP dynamic-loading discussion (github.com/orgs/modelcontextprotocol/discussions/532).
12. Counter-example implementations located by adversarial verification: Vestige (github.com/samvallad33/vestige — Bjork dual-strength, Rust MCP server); agentmemory (github.com/rohitg00/agentmemory — single-pass compress+extract).
13. Protocols: Agent Client Protocol (agentclientprotocol.com); A2A under the Linux Foundation (Apr 2026).

Changelog. v1.1 (22 July 2026): comparative-matrix rows for the two §4.11 items marked as outside the ten-cluster headline count; §4.6 supporting machinery demoted to a trailing note so the policy claim stands alone; test-suite figure refreshed with count method and as-of date. No findings, verdicts, or landscape claims changed. v1.0 (21 July 2026): initial publication.

Prepared by an orchestrated research process: three code-inspection passes over the KERNOS repository and two adversarial landscape surveys, synthesized and adjudicated by the coordinating model. All KERNOS mechanisms cited were verified against source at specific files and lines; discrepancies found during verification are disclosed in §6 rather than reconciled silently.